Linear Regression using python for E-Commerce dataset

In [1]:
```
##import libarary

import pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt
import seaborn as sn

desktop = Path.home() / "Desktop"

files = list(desktop.glob("Ecommerce Customers*"))

print(files)

df = pd.read_csv(files[0])

print(df.shape)
df.head()
```

```
[PosixPath('/Users/rajkumarsingh/Desktop/Ecommerce Customers ')]
(500, 8)
```

Out[1]:

	Email	Address	Avatar	Avg. Session Length	Time on App	Time on Website	Length of Membership	Year Amou Spe
0	mstephenson@fernandez.com	835 Frank Tunnel\nWrightmouth, MI 82180-9605	Violet	34.497268	12.655651	39.577668	4.082621	587.9510
1	hduke@hotmail.com	4547 Archer Common\nDiazchester, CA 06566-8576	DarkGreen	31.926272	11.109461	37.268959	2.664034	392.2049
2	pallen@yahoo.com	24645 Valerie Unions Suite 582\nCobbborough, D...	Bisque	33.000915	11.330278	37.110597	4.104543	487.5475
3	riverarebecca@gmail.com	1414 David Throughway\nPort Jason, OH 22070-1220	SaddleBrown	34.305557	13.717514	36.721283	3.120179	581.8523
4	mstephens@davidson-herman.com	14023 Rodriguez Passage\nPort Jacobville, PR 3...	MediumAquaMarine	33.330673	12.795189	37.536653	4.446308	599.4060

In [2]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Email                 500 non-null    object 
 1   Address               500 non-null    object 
 2   Avatar                500 non-null    object 
 3   Avg. Session Length   500 non-null    float64
 4   Time on App           500 non-null    float64
 5   Time on Website       500 non-null    float64
 6   Length of Membership  500 non-null    float64
 7   Yearly Amount Spent   500 non-null    float64
dtypes: float64(5), object(3)
memory usage: 31.4+ KB
```

In [3]:
```
df.describe()
```

Out[3]:

	Avg. Session Length	Time on App	Time on Website	Length of Membership	Yearly Amount Spent
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	33.053194	12.052488	37.060445	3.533462	499.314038
std	0.992563	0.994216	1.010489	0.999278	79.314782
min	29.532429	8.508152	33.913847	0.269901	256.670582
25%	32.341822	11.388153	36.349257	2.930450	445.038277
50%	33.082008	11.983231	37.069367	3.533975	498.887875
75%	33.711985	12.753850	37.716432	4.126502	549.313828
max	36.139662	15.126994	40.005182	6.922689	765.518462

In [4]:
```
categorical_features = df.select_dtypes(exclude="number")
categorical_features
```

Out[4]:

	Email	Address	Avatar
0	mstephenson@fernandez.com	835 Frank Tunnel\nWrightmouth, MI 82180-9605	Violet
1	hduke@hotmail.com	4547 Archer Common\nDiazchester, CA 06566-8576	DarkGreen
2	pallen@yahoo.com	24645 Valerie Unions Suite 582\nCobbborough, D...	Bisque
3	riverarebecca@gmail.com	1414 David Throughway\nPort Jason, OH 22070-1220	SaddleBrown
4	mstephens@davidson-herman.com	14023 Rodriguez Passage\nPort Jacobville, PR 3...	MediumAquaMarine
...	...	...	...
495	lewisjessica@craig-evans.com	4483 Jones Motorway Suite 872\nLake Jamiefurt,...	Tan
496	katrina56@gmail.com	172 Owen Divide Suite 497\nWest Richard, CA 19320	PaleVioletRed
497	dale88@hotmail.com	0787 Andrews Ranch Apt. 633\nSouth Chadburgh, ...	Cornsilk
498	cwilson@hotmail.com	680 Jennifer Lodge Apt. 808\nBrendachester, TX...	Teal
499	hannahwilson@davidson.com	49791 Rachel Heights Apt. 898\nEast Drewboroug...	DarkMagenta

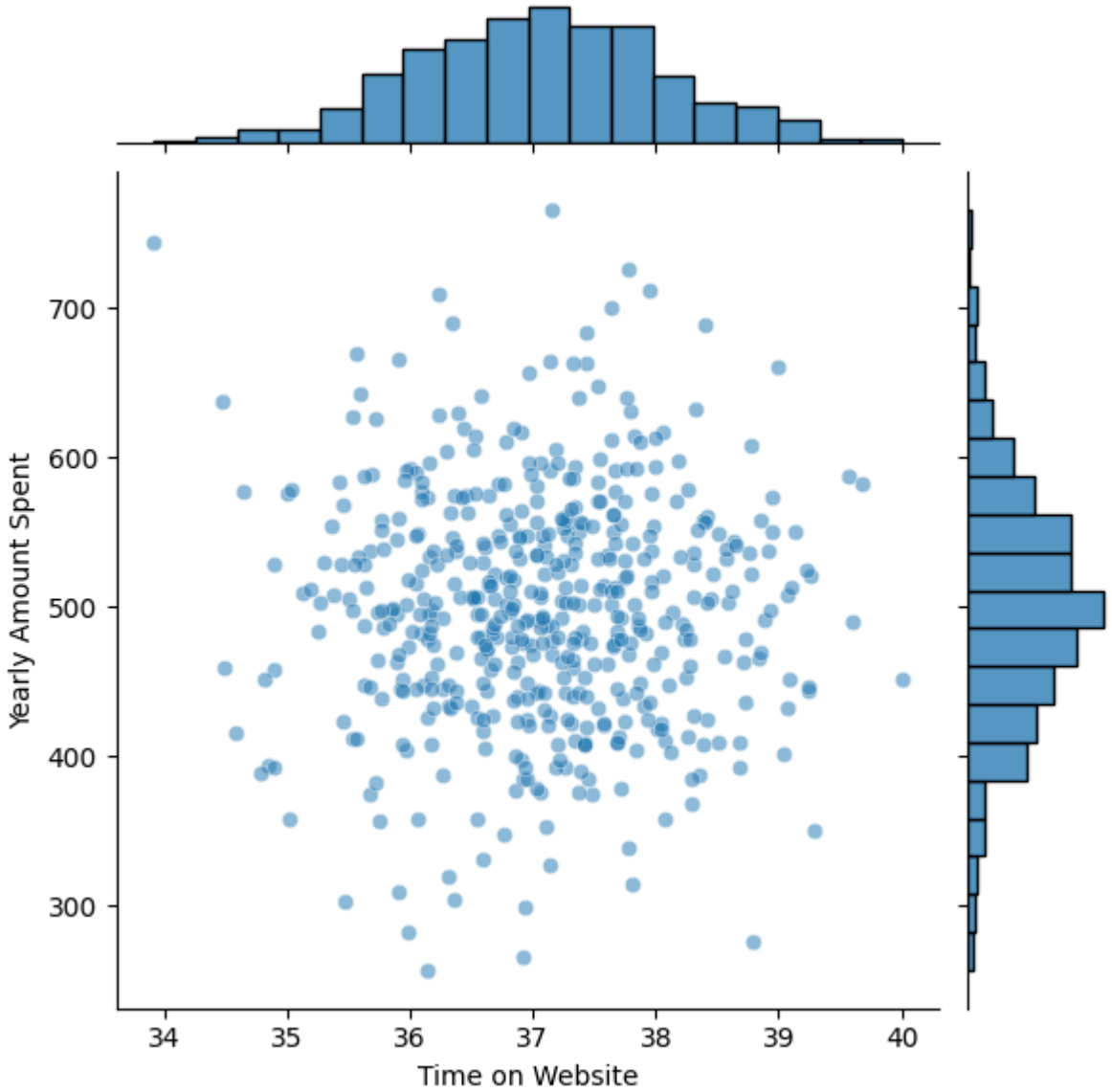
500 rows × 3 columns

In [5]:
```
Numerical_features = df.select_dtypes(exclude="object")
Numerical_features.columns
```

Out[5]:
```
Index(['Avg. Session Length', 'Time on App', 'Time on Website',
       'Length of Membership', 'Yearly Amount Spent'],
      dtype='object')
```

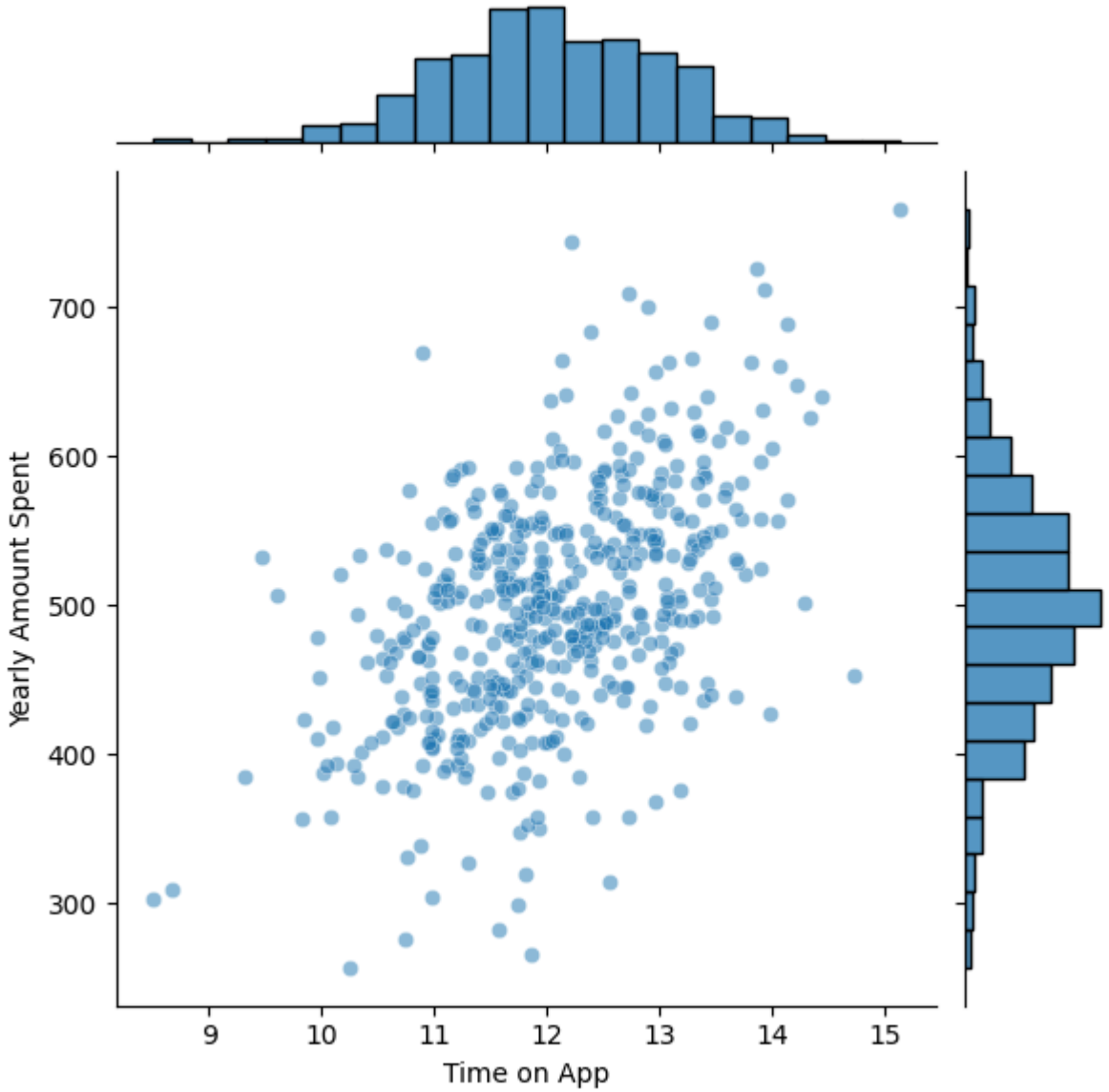
In [6]:
```
#EDA
sn.jointplot(x ="Time on Website", y = "Yearly Amount Spent", data =df, alpha =0.5)
```

Out[6]:
```
<seaborn.axisgrid.JointGrid at 0x12517d430>
```



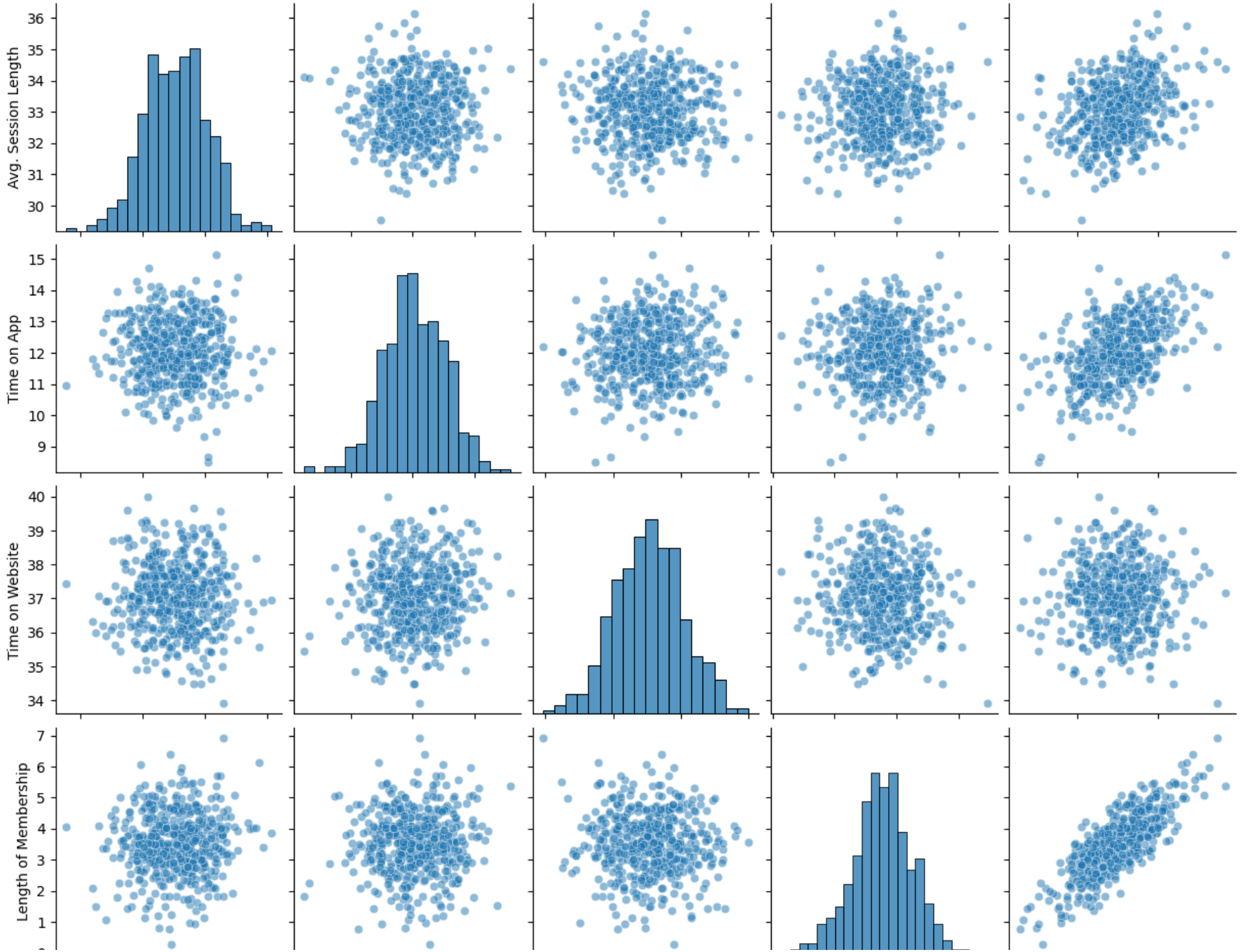
In [7]:
```
sn.jointplot(x ="Time on App", y = "Yearly Amount Spent", data =df, alpha =0.5)
```

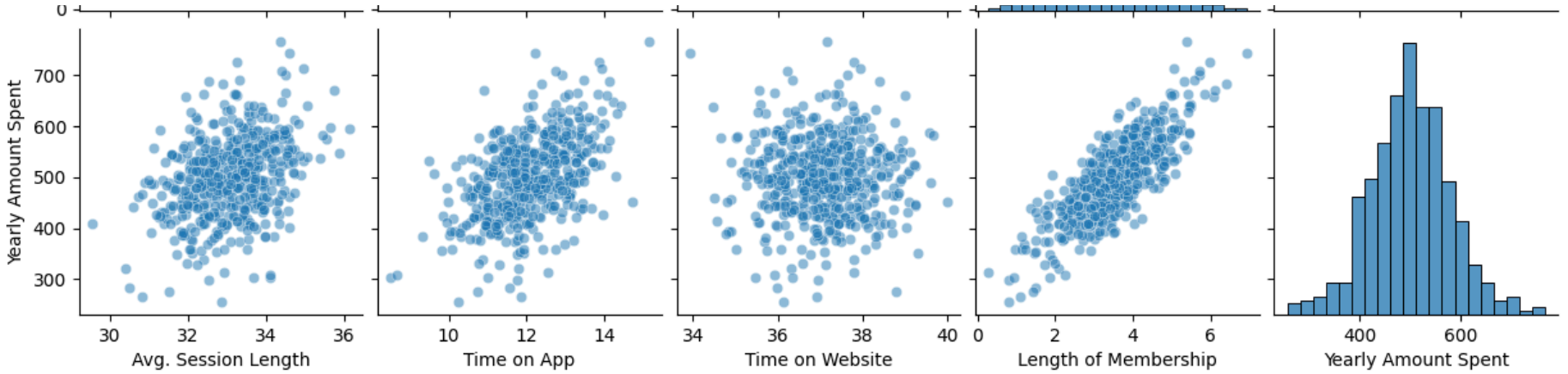
Out[7]: <seaborn.axisgrid.JointGrid at 0x12541bd00>



In [8]:
```
sn.pairplot(df, kind='scatter', plot_kws={'alpha' : 0.5})
```

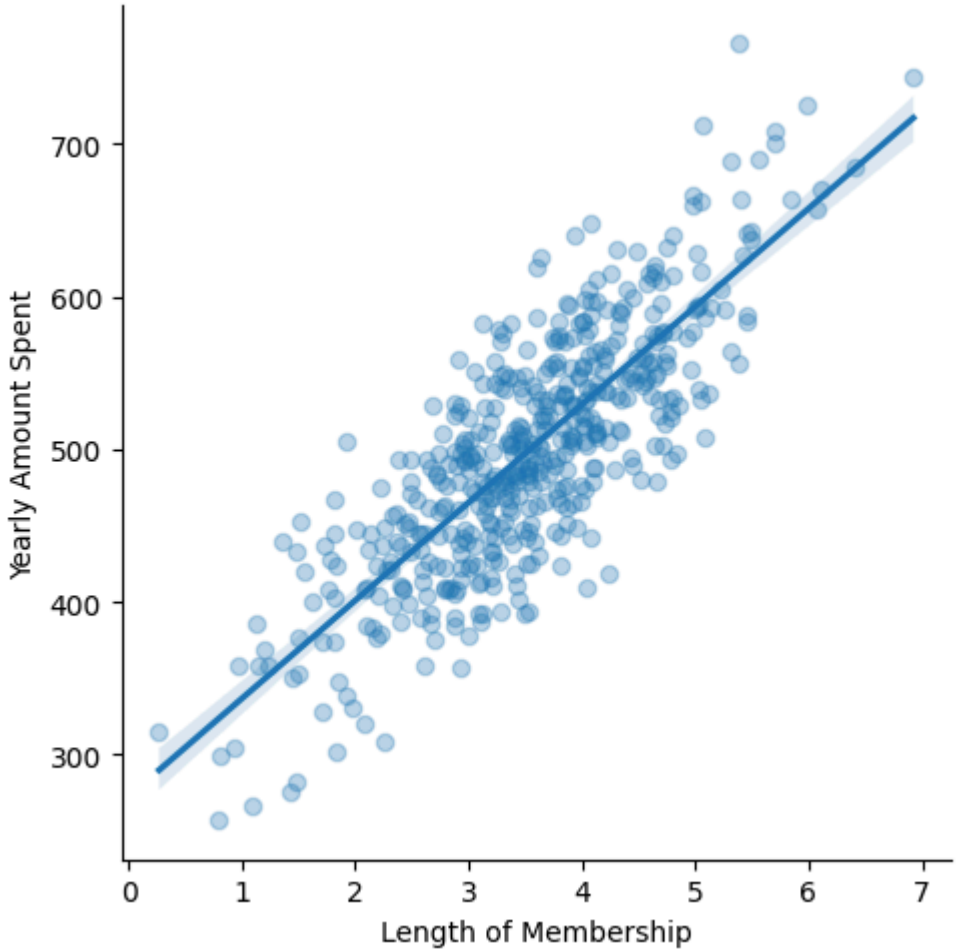
Out[8]:
```
<seaborn.axisgrid.PairGrid at 0x125602eb0>
```


Avg. Session Length
Time on App
Time on Website
Length of Membership



In [9]:
```
sn.lmplot(x = 'Length of Membership', y = "Yearly Amount Spent", data = df, scatter_kws={'alpha': 0.3})
```

Out[9]:
```
<seaborn.axisgrid.FacetGrid at 0x12624ef10>
```



In [10]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

In [11]:
```
# X = independent variables
# y = target variable

X = df[[
    "Avg. Session Length",
    "Time on App",
```

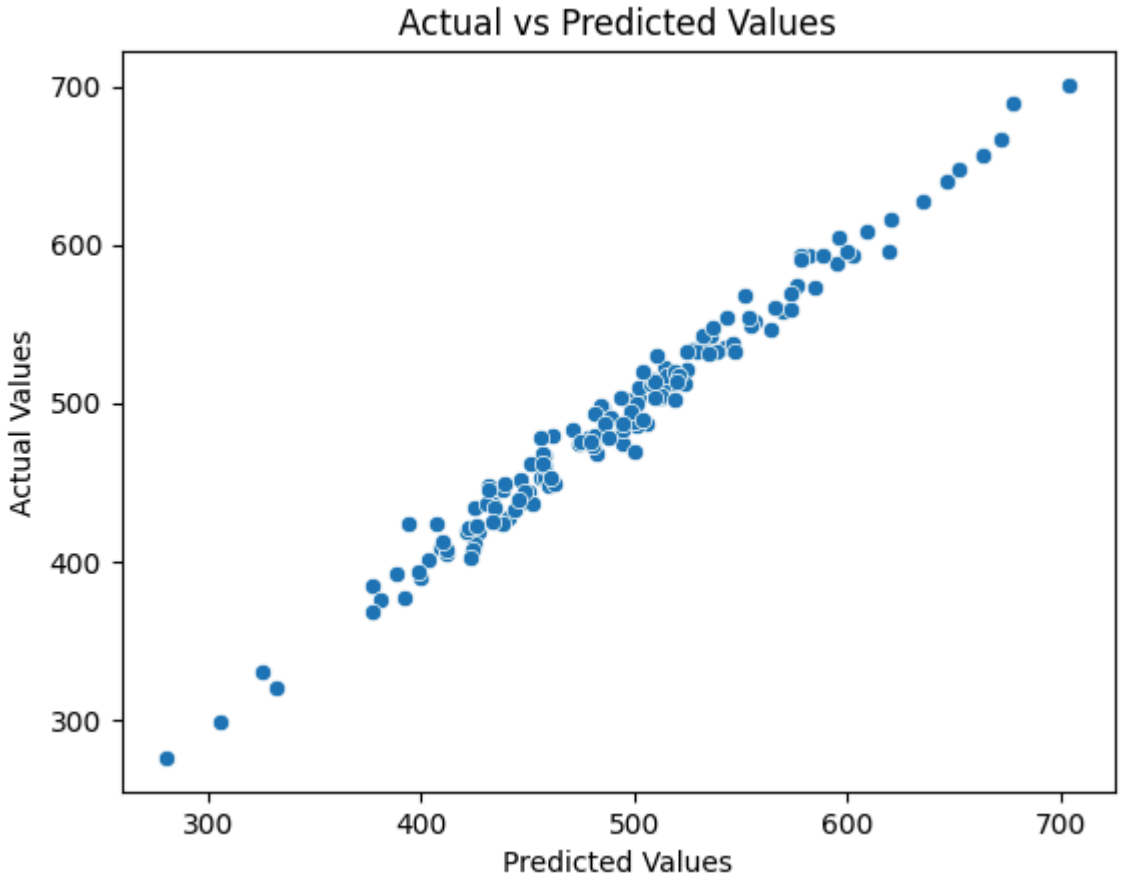
```
    "Time on Website",
    "Length of Membership"
]]

y = df["Yearly Amount Spent"]
```

In [12]:
```
# 2. Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)
```

In [13]:
```
# 3. Create the Linear Regression model
lr = LinearRegression()

# 4. Train the model
lr.fit(X_train, y_train)

# 5. Make predictions
y_pred = lr.predict(X_test)
```

In [14]:
```
sn.scatterplot(x=y_pred, y=y_test)

plt.xlabel("Predicted Values")
plt.ylabel("Actual Values")
plt.title("Actual vs Predicted Values")
```

Out[14]:
```
Text(0.5, 1.0, 'Actual vs Predicted Values')
```



In [15]:
```
lr.coef_
```

Out[15]:
```
array([25.72425621, 38.59713548,  0.45914788, 61.67473243])
```

In [16]:
```
cdf = pd.DataFrame(lr.coef_, X.columns ,columns = ["Coef"])
cdf
```

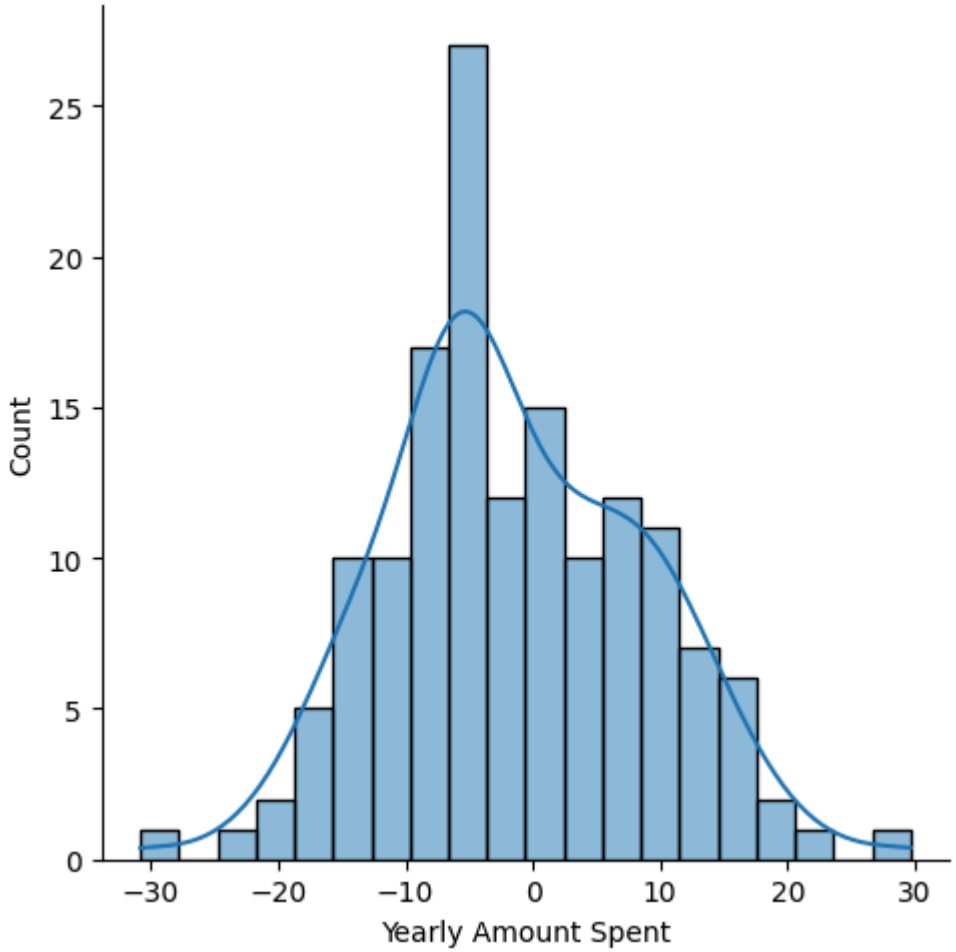
Out[16]:

	Coef
Avg. Session Length	25.724256
Time on App	38.597135
Time on Website	0.459148
Length of Membership	61.674732

In [17]:
```
#Residuals
residuals = y_test -y_pred
```

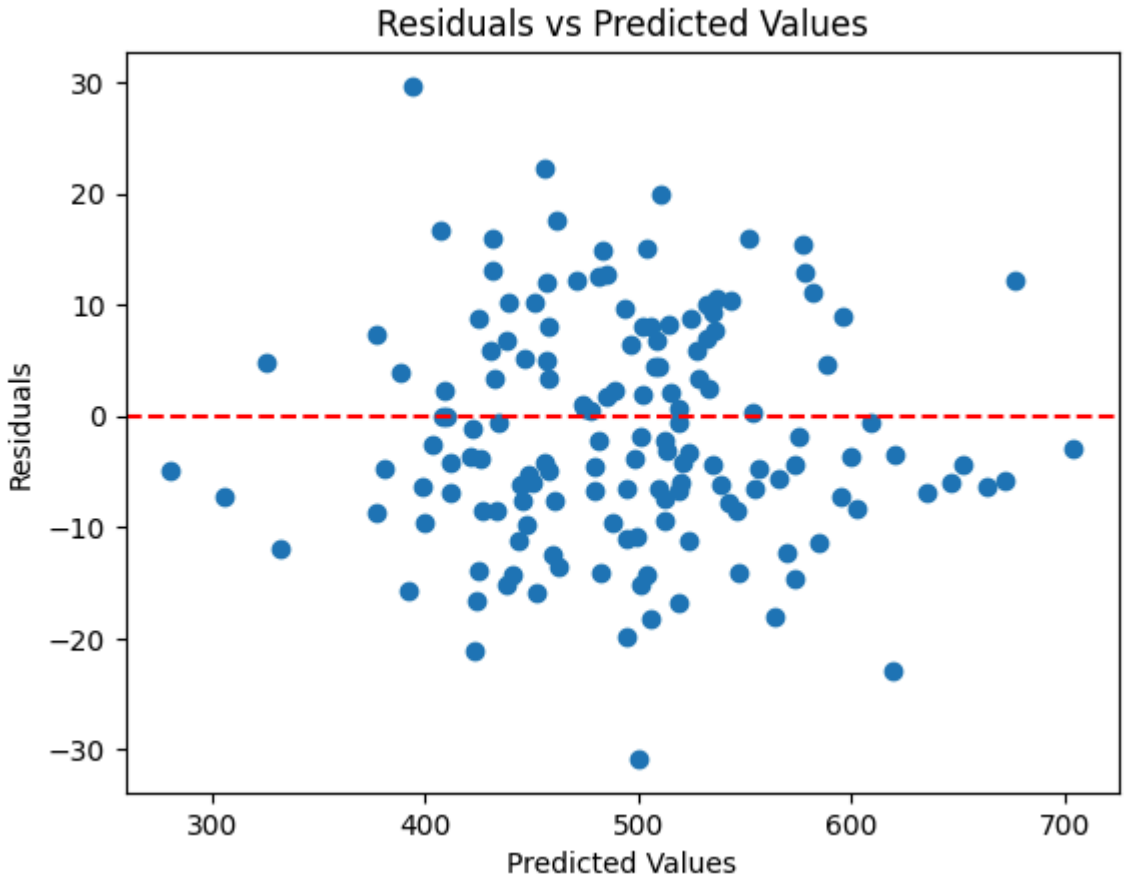
In [18]:
```
sn.displot(residuals, bins=20, kde=True)
```

Out[18]:
```
<seaborn.axisgrid.FacetGrid at 0x129aa9dc0>
```



In [21]:
```
plt.scatter(y_pred, residuals)

plt.axhline(
    y=0,
    color="red",
    linestyle="--"
)

plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Predicted Values")
```

```
plt.show()
```



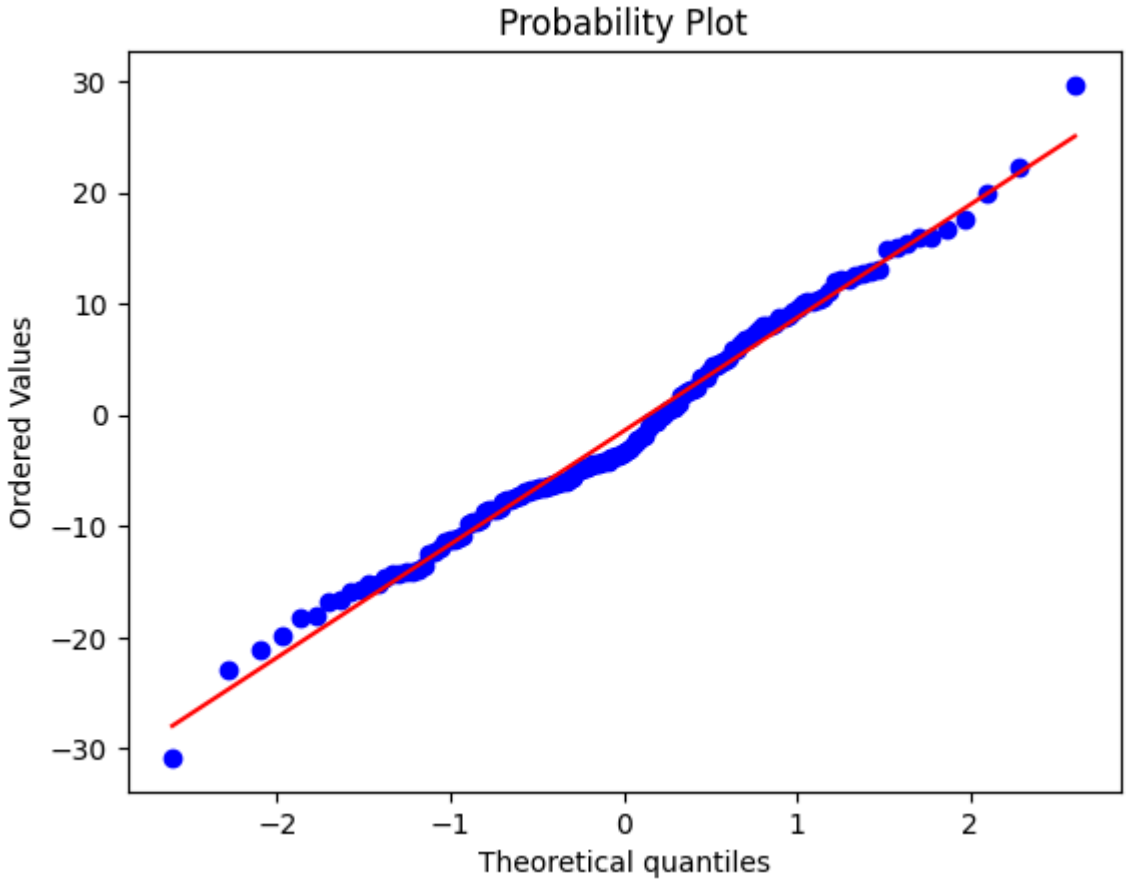
In [19]:
```
# Evaluation
print("MAE :", mean_absolute_error(y_test, y_pred))
print("MSE :", mean_squared_error(y_test, y_pred))
print("R2  :", r2_score(y_test, y_pred))
```

```
MAE : 8.426091641432048
MSE : 103.91554136503242
R2  : 0.9808757641125857
```

In [20]:
```
import pylab
import scipy.stats as stats
stats.probplot(residuals, dist="norm", plot =pylab)
```

Out[20]:
```
((array([-2.60376328, -2.283875  , -2.1005573 , -1.96875864, -1.86428437,
         -1.77691182, -1.70131573, -1.63435332, -1.57400778, -1.51890417,
         -1.46806125, -1.42075308, -1.37642684, -1.33465133, -1.29508341,
         -1.25744533, -1.22150891, -1.18708433, -1.15401181, -1.12215558,
         -1.0913992 , -1.06164202, -1.03279638, -1.00478546, -0.97754152,
         -0.95100448, -0.92512081, -0.89984257, -0.87512664, -0.85093408,
         -0.8272296 , -0.80398107, -0.78115919, -0.75873709, -0.73669013,
         -0.71499557, -0.69363244, -0.67258128, -0.65182406, -0.63134396,
         -0.61112532, -0.59115349, -0.57141472, -0.55189613, -0.53258558,
         -0.51347162, -0.49454346, -0.47579085, -0.45720409, -0.43877397,
         -0.4204917 , -0.40234892, -0.38433762, -0.36645016, -0.3486792 ,
         -0.33101768, -0.31345882, -0.29599609, -0.27862316, -0.26133393,
         -0.24412247, -0.22698303, -0.20991002, -0.19289797, -0.17594158,
         -0.15903562, -0.142175  , -0.12535471, -0.10856981, -0.09181544,
         -0.07508681, -0.05837916, -0.0416878 , -0.02500804, -0.00833524,
          0.00833524,  0.02500804,  0.0416878 ,  0.05837916,  0.07508681,
          0.09181544,  0.10856981,  0.12535471,  0.142175  ,  0.15903562,
          0.17594158,  0.19289797,  0.20991002,  0.22698303,  0.24412247,
          0.26133393,  0.27862316,  0.29599609,  0.31345882,  0.33101768,
          0.3486792 ,  0.36645016,  0.38433762,  0.40234892,  0.4204917 ,
          0.43877397,  0.45720409,  0.47579085,  0.49454346,  0.51347162,
          0.53258558,  0.55189613,  0.57141472,  0.59115349,  0.61112532,
          0.63134396,  0.65182406,  0.67258128,  0.69363244,  0.71499557,
          0.73669013,  0.75873709,  0.78115919,  0.80398107,  0.8272296 ,
          0.85093408,  0.87512664,  0.89984257,  0.92512081,  0.95100448,
          0.97754152,  1.00478546,  1.03279638,  1.06164202,  1.0913992 ,
          1.12215558,  1.15401181,  1.18708433,  1.22150891,  1.25744533,
          1.29508341,  1.33465133,  1.37642684,  1.42075308,  1.46806125,
          1.51890417,  1.57400778,  1.63435332,  1.70131573,  1.77691182,
          1.86428437,  1.96875864,  2.1005573 ,  2.283875  ,  2.60376328]),
  array([-30.81219   , -22.907004  , -21.17779119, -19.90776028,
         -18.22493961, -18.03964213, -16.77823692, -16.53475038,
         -15.83190809, -15.62749971, -15.13789844, -15.1228114 ,
         -14.54056593, -14.26974236, -14.22177477, -14.16409746,
         -14.05738994, -13.94275933, -13.47756757, -12.39225674,
         -12.32616303, -12.01395095, -11.40750047, -11.27905389,
         -11.14494682, -11.05474067, -10.91236301,  -9.76859176,
          -9.65579326,  -9.53909902,  -9.35140986,  -8.77533735,
          -8.57539695,  -8.46317449,  -8.42555803,  -8.24581704,
          -7.80037479,  -7.658202  ,  -7.54723593,  -7.37571535,
```

```
        -7.25024405,  -7.21135062,  -6.94170975,  -6.84797161,
        -6.6900814 ,  -6.66728641,  -6.60373256,  -6.50076449,
        -6.43777911,  -6.28822479,  -6.25972253,  -6.23679081,
        -6.22783768,  -6.0217341 ,  -5.99058094,  -5.95781391,
        -5.87417226,  -5.65260027,  -5.32404755,  -4.96586144,
        -4.84916247,  -4.7844326 ,  -4.7443785 ,  -4.60868617,
        -4.37183172,  -4.37009922,  -4.31671088,  -4.20086514,
        -4.13783869,  -4.10407742,  -3.89281919,  -3.78921913,
        -3.64843414,  -3.55815317,  -3.53110675,  -3.26933312,
        -3.13635707,  -2.91324727,  -2.63679547,  -2.29778621,
        -2.17097803,  -1.91799525,  -1.916866  ,  -1.04288155,
        -0.62045517,  -0.56774014,  -0.50299299,  -0.09851875,
        -0.05182453,   0.2648391 ,   0.47852893,   0.64733189,
         0.75866389,   1.03803604,   1.68008398,   1.9537748 ,
         2.11538318,   2.23822695,   2.33091007,   2.4237775 ,
         3.30791927,   3.37774597,   3.41448206,   3.94701636,
         4.39108997,   4.42457152,   4.54204665,   4.74821467,
         5.06022067,   5.16197465,   5.87016501,   5.89816812,
         6.50064042,   6.71445214,   6.78413309,   7.00766112,
         7.40259937,   7.68552217,   8.04004857,   8.06183019,
         8.07697354,   8.29025588,   8.68466916,   8.82214723,
         8.93054295,   9.39906831,   9.60228333,   9.94777859,
        10.16425625,  10.2708491 ,  10.32717561,  10.61183903,
        11.19842685,  11.95752326,  12.11362618,  12.18601994,
        12.48279146,  12.68810576,  12.83920766,  13.1121931 ,
        14.8396788 ,  15.02798444,  15.51951594,  15.91218704,
        15.92819645,  16.7359091 ,  17.65947592,  19.84259442,
        22.31734843,  29.66416022])),
 (np.float64(10.17978592199497),
  np.float64(-1.447120411094994),
  np.float64(0.9944627992644606)))
```


Probability Plot
Ordered Values
Theoretical quantiles
30
20
10
0
−10
−20
−30
−2
−1
0
1
2

In [ ]: